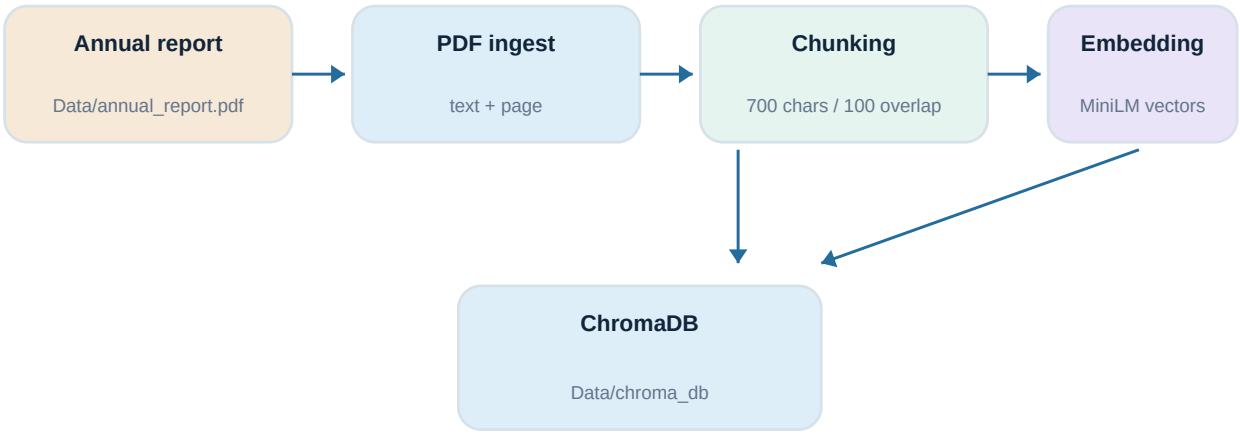
PROJECT USER & DEVELOPER MANUAL

Infosys Annual Report Analyst

A practical guide to preparing the report index, running the Streamlit agent, understanding the request flow, and maintaining every project file.



One-time or occasional preparation: main.py builds the searchable local index.

Preparation flow at a glance

What this project does

This application is a retrieval-augmented generation (RAG) assistant for the Infosys Integrated Annual Report 2023-24. It extracts text from a local PDF, stores searchable text passages and vectors in a local ChromaDB database, then retrieves relevant passages for each question and asks Gemini to produce a streamed answer with report page references.

The report PDF, local vector index, and API key are intentionally kept out of the public GitHub repository. A new user must supply these locally before running the full agent.

Contents

Contents 2

1. Project overview 3

2. Project files and folders 5

3. First-time setup on Windows / PyCharm 5

4. Build the search index 6

5. Start and use the agent 6

6. What happens after you submit a question 7

7. File-by-file code walkthrough 8

8. Configuration files and dependencies 10

9. Troubleshooting 10

10. GitHub, cloning, and deployment 11

11. Maintenance and safe working habits 11

Appendix A. Glossary 12

1. Project overview

The project has two separate phases. Index preparation is run when the report changes or the index needs rebuilding. Question answering is run through the web interface and reuses that index.

Phase	What happens	Main entry point
Prepare index	Read the PDF, split text into page-labelled passages, create local vectors, write records into ChromaDB.	python main.py
Ask questions	Embed the question, retrieve the closest passages, send them with a grounded prompt to Gemini, stream answer and pages.	python -m streamlit run app.py

System architecture

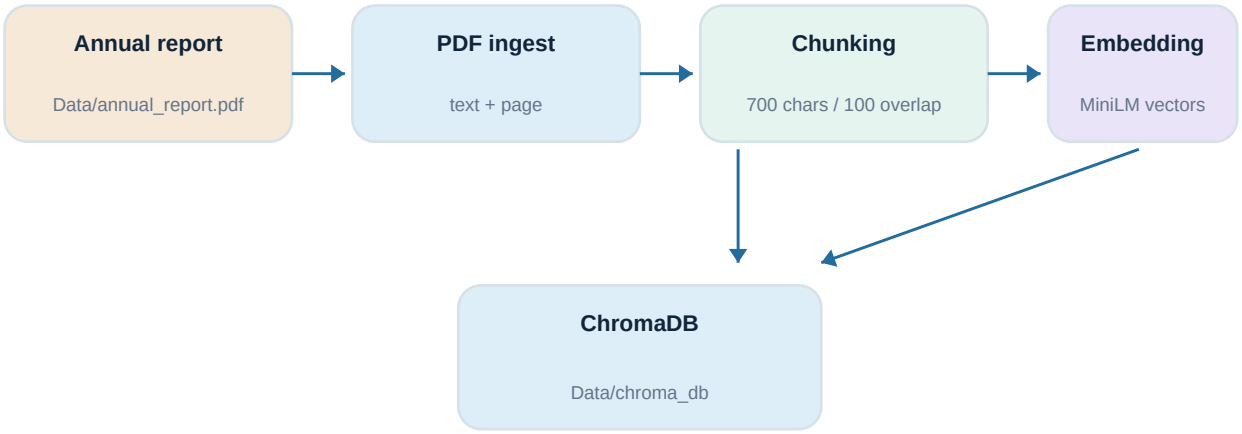
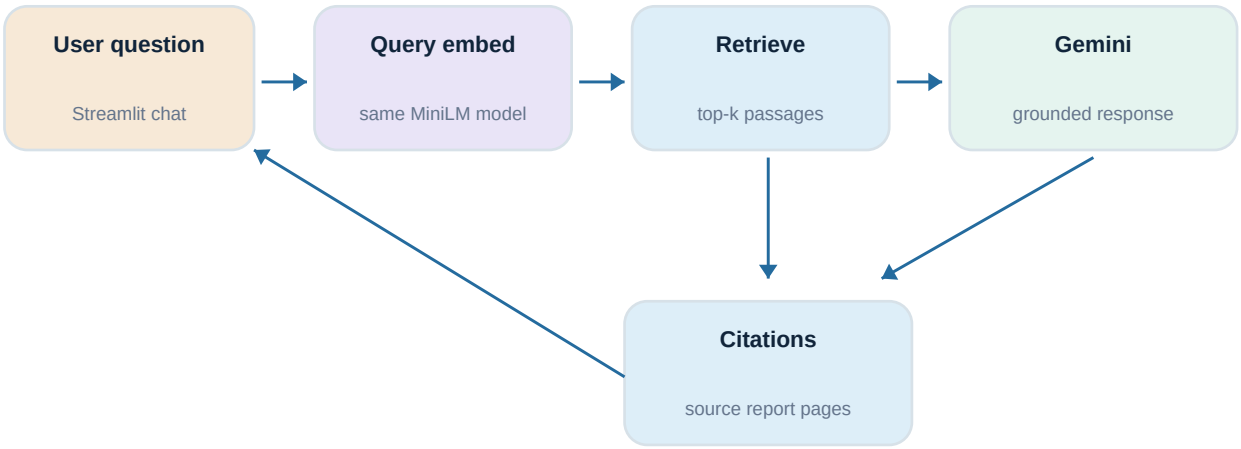


Figure 1. Offline/local index preparation. The embedding model runs locally after its first download.



Runs for each question: retrieval is local; answer generation uses the Gemini API.

Figure 2. Online question flow. Gemini is called only when a user submits a question.

Key distinction: ChromaDB stores passages and vectors; Gemini does not search the PDF itself. Retrieval selects evidence first, and the selected evidence is then supplied to Gemini.

2. Project files and folders

Path	Purpose
app.py	Streamlit interface: status panel, chat, suggested questions, evidence pages, and streaming response.
main.py	Thin command-line entry point that delegates index creation to src/Vector_store.py.
requirements.txt	Python packages installed into the project's virtual environment.
README.md	Setup, indexing, running, and deployment instructions.
.gitignore	Files kept out of Git: secrets, local report/index, virtual environments, caches, IDE settings.
src/ingest.py	Extracts text from the PDF and creates overlapping page-aware chunks.
src/Embed.py	Loads the sentence-transformer model and generates one vector per chunk.
src/Vector_store.py	Creates or updates the ChromaDB collection in bounded batches; supports --rebuild.
src/rag_pipeline.py	Loads credentials/config, retrieves relevant passages, builds the grounded prompt, streams Gemini output.
test_rag.py	Manual end-to-end smoke check; requires an index and a working Gemini API key.
Data/annual_report.pdf	Local input report. Not present in the public repository; add it yourself.
Data/chroma_db/	Generated local vector database. Rebuild with main.py; do not copy into Git.
.env or key.env	Local API key and optional model settings. Never commit or share this file.
.venv/	Project-specific Python environment. Re-create from requirements.txt on another computer.
.idea/	PyCharm project preferences; optional local IDE files, excluded from the public repository.
.git/	Git history and remote configuration. It is internal repository metadata, not application code.
src/__pycache__/	Python bytecode cache created automatically; safe to delete and ignored by Git.

3. First-time setup on Windows / PyCharm

Start from the public GitHub repository or the current project folder. The steps below assume PowerShell and a Python 3.11+ interpreter.

Step 1 - Open the project

In PyCharm, choose **Open** and select the project root folder. Confirm the interpreter points to this project's `.venv` rather than a global Python.

Step 2 - Create a virtual environment

Open the PyCharm terminal at the project root and run the command below. The environment keeps this project's packages separate from other Python projects.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Step 3 - Install packages

Install the package list from requirements.txt while the virtual environment is active.

```
python -m pip install -r requirements.txt
```

Step 4 - Add the report

Create the Data folder if needed, then place the authorized report file at exactly `Data/annual_report.pdf`. The file is intentionally not hosted in Git.

Step 5 - Configure Gemini

Create a root-level .env file using a text editor. Put the key name and your own secret value in it. Never paste the real key into this manual, source code, GitHub, screenshots, or chat.

```
GEMINI_API_KEY=put_your_own_key_here
# Optional:
GEMINI_MODEL=gemini-flash-latest
EMBEDDING_MODEL=all-MiniLM-L6-v2
```

PowerShell execution policy may prevent environment activation. In that case, use the virtual-environment executable directly: `.venv\Scripts\python.exe -m pip install -r requirements.txt` and `.venv\Scripts\python.exe -m streamlit run app.py`.

4. Build the search index

Run index preparation once after adding the report. It takes longer on first run because sentence-transformers downloads the model. The model is cached locally afterwards.

```
python main.py
```

What you see	Meaning
Reading annual_report.pdf	PyMuPDF opened the file and is extracting selectable text page by page.
Prepared N passages	PDF text was divided into 700-character chunks with 100-character overlap.
Embedding progress	all-MiniLM-L6-v2 converts every chunk into a numeric vector; batch size is 64.
Indexed N/N passages	Vector, passage text, deterministic ID, and report page metadata were written to ChromaDB in batches of 128.

Rebuild after replacing the report

Use `python main.py --rebuild` when the PDF content changes or the existing collection is inconsistent. The program first extracts and embeds all text, then deletes the old collection and writes a fresh index. Keep a copy of the PDF; the ChromaDB database can be regenerated from it.

The normal command uses upsert, so repeating it with the same PDF updates matching chunk IDs. If the new report has fewer chunks, stale records from the older report can remain; use --rebuild for a different PDF.

5. Start and use the agent

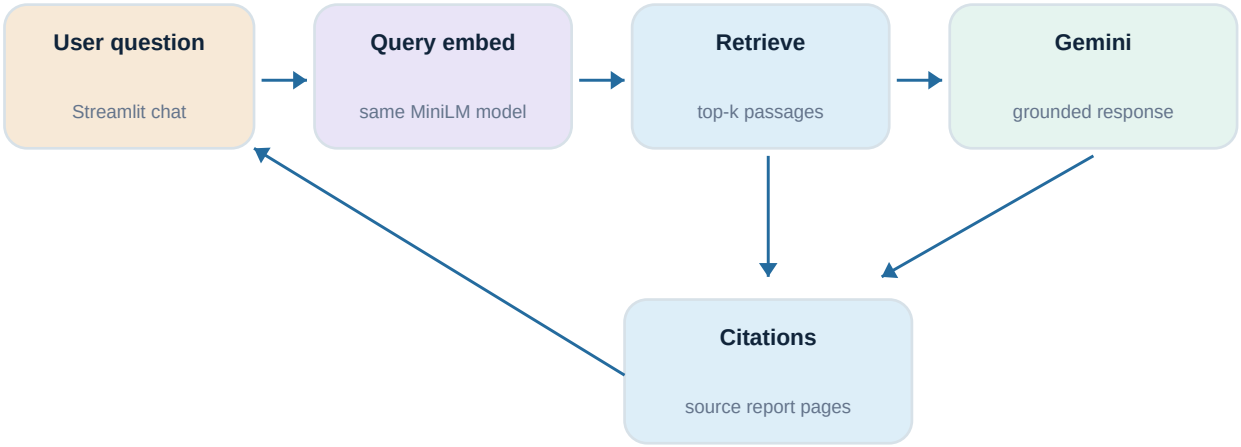
```
python -m streamlit run app.py
```

Streamlit prints a local address, usually http://localhost:8501. Open that address in a browser. The app checks whether the index and Gemini key appear available in the sidebar. It does not call Gemini when it first opens; API generation starts only after a question is submitted.

Interface element	How to use it
Evidence passages slider	Choose how many nearest report passages (top-k) to retrieve. Larger values provide more context but can add noise and increase prompt size.
Suggested questions	Click a prompt to submit it immediately. The examples cover financial results, risks, and people metrics.
Chat input	Ask a concise question. Use the scope/year/report name when it helps the search.
Agent status	Shows the search, evidence, and drafting stages. The final answer streams into the assistant message.
Report pages	The page list is generated from the retrieved Chroma metadata. The model is also instructed to cite relevant pages inline.
Clear conversation	Clears only the current browser session's displayed chat history; it does not delete the index or report.

Example questions: 'How did revenue and net profit change year over year?', 'What risks did the auditors highlight?', or 'What were employee count and attrition?'

6. What happens after you submit a question



Runs for each question: retrieval is local; answer generation uses the Gemini API.

Detailed sequence

Step	Component	Action
1	app.py	Stores and displays the user's prompt in Streamlit session state.
2	rag_pipeline.py	Checks that the question is non-empty, reads the API key, opens the Chroma collection, and limits top-k to the available record count.
3	SentenceTransformer	Encodes the question using the same embedding model used during indexing.
4	ChromaDB	Finds the closest vectors and returns passage text plus page metadata.
5	Prompt builder	Labels retrieved text with page numbers and tells the model to use only supplied evidence and cite pages.
6	Gemini SDK	Starts generate_content_stream; text chunks are yielded as they arrive.
7	Streamlit	Displays the stream and source page list, then saves the answer in the current session history.

This is a RAG agent-style workflow with retrieval and synthesis stages. It currently has no autonomous tools, planning loop, or external actions; it answers from retrieved report excerpts.

7. File-by-file code walkthrough

app.py - user interface

Imports Streamlit and four helpers/constants from src.rag_pipeline. set_page_config sets the browser title, icon, wide layout, and open sidebar. The markdown block adds the dark agent dashboard style and header; unsafe_allow_html is used only for this fixed page markup and locally generated page-number text.

Code area	What it does
Sidebar controls	Slider sets top_k. get_collection() and count() provide index status. get_api_key() shows whether a key is configured without displaying it. Buttons place a sample prompt into session state or clear the chat.
Session history	Initializes st.session_state.messages once, then redraws each saved user and assistant message on Streamlit reruns.
Chat input	Reads a typed question or pending suggested prompt.
Answer block	Calls ask_annual_report_stream, displays evidence pages, then passes the generator to st.write_stream.
Error handling	Shows a safe generic error and exception type; it does not print exception text that could contain sensitive provider details.
Final append	Saves assistant content and pages so they remain visible during the current session.

main.py - index command entry point

Imports main() from src.Vector_store and calls it only when the file is executed as a script. This is the command used by python main.py. The imported function parses --rebuild and orchestrates indexing.

```
from src.Vector_store import main

if __name__ == "__main__":
```

```
main()
```

src/ingest.py - PDF extraction and chunking

PROJECT_ROOT is calculated from the file's location rather than a fixed drive path. PDF_PATH points to Data/annual_report.pdf. ingest_annual_report validates that the PDF exists and that chunk_size/overlap are sensible. It opens the PDF in a context manager, extracts sorted text from each page, normalizes whitespace, skips blank pages, and emits dictionaries containing page and text. Default chunks are 700 characters with 100 characters of overlap, with a nearby whitespace boundary preferred to avoid splitting words.

Implementation note: the local variable named step is calculated but unused; actual movement is governed by end - overlap. The code still applies the intended 100-character overlap.

src/Embed.py - vector generation

load_embedding_model() caches one SentenceTransformer instance with lru_cache. generate_embeddings() calls the PDF ingest function, sends all passage strings to model.encode in batches of 64, shows progress, and adds each vector as a plain Python list under the vector field. The vector is later used by ChromaDB, while page and text become searchable metadata/content.

src/Vector_store.py - local ChromaDB index

setup_vector_database(rebuild=False, batch_size=128) obtains embedded chunks, creates Data/chroma_db, and opens a persistent Chroma client. With --rebuild it deletes the named collection; otherwise it gets or creates the collection. Each batch is upserted with a deterministic ID, passage text, integer page metadata, and vector. The collection count is printed at completion. main() uses argparse to expose the --rebuild switch. The import Path is currently unused; it does not affect operation.

src/rag_pipeline.py - retrieval and generation

Name	Role
PROJECT_ROOT / DB_PATH	Derives local paths from this source file so the project can move between folders or computers.
COLLECTION_NAME	Must match the collection created by src/Vector_store.py.
Model settings	Read from environment variables. Defaults: all-MiniLM-L6-v2 for embeddings, gemini-3.6-flash as the primary answer model, and gemini-3.8-flash as the fallback.
get_api_key()	Checks GEMINI_API_KEY in the process environment, then Streamlit secrets. It returns the key to the SDK and never displays it.
get_collection()	Initializes the persistent Chroma client and collection once; a lock avoids duplicate initialization across concurrent calls.
get_embedding_model()	Initializes and retains the sentence-transformer model once per process.
ask_annual_report_stream()	Validates input/key/index, embeds the question, retrieves up to top_k passages, builds page-labelled context and a grounded prompt, then returns a streaming generator and sorted source pages. The generator keeps the Gemini client open until streaming finishes and tries the fallback model if the primary fails before producing text.

test_rag.py - manual smoke check

The run_test function asks one fixed question, prints the streamed response and report pages, and runs only under the Python script guard. Run it with python test_rag.py after indexing and configuring the key. It makes a real Gemini request; it is not an offline unit-test suite.

8. Configuration files and dependencies

File / package	Purpose in this project
requirements.txt	Declares Streamlit (interface), google-genai (Gemini API), ChromaDB (persistent vector store), protobuf (dependency compatibility), sentence-transformers (local embedding model), PyMuPDF (PDF extraction), and python-dotenv (.env loading).
.gitignore	Ignores .env/key.env and Streamlit secrets; .venv/venv; Python bytecode/cache; IDE settings; the PDF; and generated ChromaDB data.
README.md	Project quick start and repository/deployment overview. This manual expands on it with complete workflow and code explanations.
.env	Local-only key and optional settings. The value must never be committed.
.idea/	PyCharm metadata; not needed to run from another IDE.
.venv/	Installed packages. Recreate with requirements.txt rather than copying this folder to another computer.

The dependency versions in requirements.txt use version ranges for some packages. For reproducible production deployments, consider pinning a tested full dependency lock file.

9. Troubleshooting

Symptom	Likely cause	What to do
Annual report PDF not found	PDF missing or filename/path differs.	Place the file at Data/annual_report.pdf, then run python main.py.
Index not found / collection missing	The local Chroma index was not built or was removed.	Run python main.py and wait for the final index-ready message.
GEMINI_API_KEY is not configured	Key absent from process env, .env, key.env, or Streamlit secrets.	Set it locally and restart Streamlit. Do not put it in source code or GitHub.
No response / model error	Network, account access, model availability, quota, or API-key issue.	Check internet, billing/quota, key validity, and optional GEMINI_MODEL setting.
Database is locked / readonly	Another process may be rebuilding the persistent store or the folder lacks write permission.	Stop duplicate app/index processes; ensure Data/chroma_db is writable; run the app from the project folder.
Citations seem off	Retrieval returned related but insufficient passages or the report's printed page differs from PDF page numbering.	Increase Evidence passages, ask a narrower question, and verify cited evidence against the source report.
Rebuild is slow	PDF extraction, model initialization, and embedding every page are CPU/memory intensive.	Wait for the progress indicator; first run also downloads the model. Avoid rebuilding unless needed.

10. GitHub, cloning, and deployment

The public repository contains source and setup instructions, not the annual report, its generated index, or API credentials. This is deliberate: the report may be subject to distribution terms, the index is reproducible, and a key must remain private.

```
git clone https://github.com/saksham146/infosys-annual-report-analysis.git
cd infosys-annual-report-analysis
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

After cloning, add the report file and local API key, build the index with python main.py, then run Streamlit. For Streamlit Community Cloud, configure GEMINI_API_KEY in the deployment's Secrets panel and provide a valid report/index provisioning step; the local Chroma directory is intentionally not uploaded.

The current local checkout has a publish remote named annual-report-publish. The public repository's main branch begins at a clean root commit and does not inherit the previous repository's history.

11. Maintenance and safe working habits

- Keep the input PDF at the expected path, and rebuild the local collection when changing the report.
- Never commit .env, key.env, Streamlit secrets, the PDF, the Chroma database, or .venv.
- If a key is accidentally exposed, revoke it in the provider console and create a replacement; deleting a file from the latest commit does not remove it from earlier Git history.
- Use the same embedding model name for indexing and question queries. Changing EMBEDDING_MODEL requires rebuilding the index with that model.
- Keep the Streamlit app stopped while replacing or rebuilding the Chroma database to avoid competing access to its files.
- Treat generated answers as report-grounded assistance; confirm material financial figures and context against the cited report pages.

Scope today: one report, local PDF indexing, vector search, and Gemini answers. It has no multi-document upload, model conversation memory, fine-tuning, authentication layer, or autonomous actions.

Appendix A. Glossary

Term	Plain-language meaning
RAG	Retrieval-augmented generation: find relevant source text first, then ask a language model to answer from it.
Chunk / passage	A smaller text excerpt from a PDF page that can be searched independently.
Embedding / vector	A list of numbers representing the meaning of text for approximate similarity search.
Top-k	The maximum number of nearest passages returned for a question.
Metadata	Extra fields stored with a passage; here the main field is the PDF page number.
Stream	The answer is delivered in pieces as Gemini generates it, rather than waiting for one complete response.
Upsert	Insert a record if it is new, or update it if its ID already exists.